EVERFIT · AI TEAM

Take-Home Exercise — AI Engineer

Mid-Senior Level | Estimated 6–8 hours | AI tools allowed & expected | March 2026

Overview

Build an AI Workout Coach — an intelligent assistant that answers fitness questions from a knowledge base, analyzes a user's training history, assists coaches with multi-step tasks through an agent, and handles all of this safely in a health-adjacent context.

This exercise evaluates: RAG pipeline design, LLM engineering, agent architecture, evaluation methodology, guardrails thinking, and production readiness.

Time estimate: 6–8 hours. Before you start, provide your own time estimate. We want to see how you scope work, not just how fast you code.

Context

Everfit is a B2B2C fitness coaching platform. Coaches create workout programs for their clients and communicate with them daily. We are building AI features that help coaches and clients get smarter insights from their training data — while keeping the coach's voice and judgment at the center of every interaction.

Features

Feature 1 — Fitness Knowledge RAG (Required)

Build a RAG pipeline that answers fitness-related questions using a provided knowledge base (~20 markdown documents). You may supplement with your own curated content — document what you added and why.

The system must:

- ✓ Ingest documents, chunk them, and store embeddings in a vector database
- ✓ Accept a natural language question via API and return a grounded answer
- ✓ Include source references (which document/chunk was used) in the response
- ✓ Handle out-of-scope questions gracefully — "What's the weather today?" must not produce a fitness answer

Guardrails Requirement

The system must refuse or redirect questions that could constitute medical advice — including injury rehabilitation without professional assessment, eating disorder-risk content, and medical diagnosis requests.

Document your refusal strategy:

- What signals trigger a refusal
- What message does the user receive
- How do you avoid being so restrictive that legitimate fitness questions get blocked

Include at least 2 adversarial test cases targeting this boundary in your evaluation set.

Feature 2 — Workout History Analysis (Required)

Build an endpoint that accepts a user's workout history (JSON array) and a natural language question, then returns an AI-generated insight.

Example workout entry:

```
{  "date": "2026-03-20",  "exercise": "Bench Press",  "sets": [    { "reps": 10,
"weight": 80, "unit": "kg" },    { "reps": 8,  "weight": 85, "unit": "kg" },    {
"reps": 6,  "weight": 90, "unit": "kg" }  ]}
```

Example questions the system should handle:

- "What's my bench press trend over the last month?"
- "Which exercises am I neglecting?"
- "Am I overtraining chest compared to back?"
- "Suggest a workout plan for next week based on my history"

The system must:

- ✓ Parse and analyze workout data before passing to the LLM — do not dump raw JSON into the prompt
- ✓ Provide data-backed responses (reference specific numbers, dates, trends)
- ✓ Handle edge cases: empty history, insufficient data, unknown exercises
- ✓ Maintain user data isolation — User A's history must never appear in User B's context or response. Include at least 1 test case verifying this boundary.

Feature 3 — Coach Assist Agent

Build a simple agent that helps a coach answer a multi-step question by deciding which tools to use and in what order.

The agent must have access to at least two tools:

- rag_search(query) — your Feature 1 RAG pipeline
- analyze_history(user_id, question) — your Feature 2 analysis endpoint

Example multi-step questions the agent should handle:

- "Based on John's recent workout history, is he ready to increase bench press weight? What does proper progressive overload look like for his current level?"
- "My client hasn't done any pulling exercises this month and is complaining of shoulder tightness. What should I tell her?"

The agent must:

- ✓ Decide which tools to call and in what sequence — do not hardcode the call order
- ✓ Produce a single coherent response that cites both data sources when both are used
- ✓ Handle the case where one tool returns insufficient data gracefully

Document in your README:

- What happens if the agent calls the wrong tool first?
- How would you add a third tool without rewriting the agent logic?
- What's the failure mode you're most worried about in production?

Note: You do not need a full agentic framework (LangGraph, CrewAI, etc.). A clean implementation using function/tool-calling via the LLM API is sufficient. If you choose to use a framework, justify why.

Feature 4 — Evaluation Pipeline (Required)

Build an evaluation system that measures the quality of your RAG, analysis, and agent outputs.

Requirements:

- ✓ Create a test set of at least 15 question-answer pairs: 5 for RAG, 5 for workout analysis, 3 for the agent, 2 adversarial cases targeting guardrails
- ✓ Implement at least 3 evaluation metrics — one must be an LLM-as-judge metric (e.g. faithfulness, tone quality), one must be rule-based (e.g. source attribution present, data values referenced)
- ✓ Run the evaluation and include results in your submission
- ✓ Write an honest analysis: what failed, what surprised you, what you would change

EVALUATION.md must include:

- Your full test set
- Evaluation results per metric
- At least 2 specific failure examples with root cause analysis
- What you would improve in the next iteration

Technical Constraints

Constraint	Detail
Language	Python (FastAPI recommended, but your choice — justify it)
LLM	Any model — OpenAI, Anthropic, open-source (Ollama/vLLM), or a combination. Justify your choice including cost and latency tradeoffs.
Vector DB	Any (Chroma, Pinecone, Milvus, pgvector, etc.) — justify your choice
Local setup	Must run locally. A single docker compose up is strongly preferred.
API keys	Via environment variables only — no hardcoded secrets

What We Evaluate

Criteria	What We Look For
RAG quality	Chunking strategy, retrieval accuracy, prompt construction, source attribution, hallucination handling
LLM engineering	Prompt design, structured output handling, token efficiency, model selection reasoning
Agent design	Tool selection logic, multi-step reasoning, graceful degradation when tools return poor results
Data processing	How workout data is pre-processed before LLM — aggregation, trend detection, statistical analysis vs raw dump
Guardrails	Refusal strategy quality, adversarial test cases, avoiding over-restriction
Evaluation rigor	Quality of test cases, LLM-as-judge implementation, honest failure analysis
Production thinking	Error handling, latency awareness, cost per query estimate, data isolation, logging
Architecture	Clean separation between retrieval, processing, and generation layers. Extensibility.
Code quality	Clean code, meaningful tests, documentation
AI adoption	See below — weighted equally with technical output

AI Adoption Evaluation (Critical — weighted equally)

We are an AI-first team. We evaluate how you use AI tools as part of your development workflow — not whether you used them, but how well you direct, correct, and build judgment on top of them.

AI_WORKFLOW.md (Required)

- Tools used — which AI tools at each stage and for what specific purpose
- At least 2 examples where AI output was wrong or suboptimal — what it produced, why it was wrong, exactly how you corrected it
- At least 1 example where you rejected an AI suggestion entirely — what it suggested, why you rejected it, what you did instead
- Your prompting strategy — full context upfront? Smaller prompts? System prompts or rules files? How did your approach evolve?
- At least 1 reflection on the guardrails implementation — did AI tools help or hinder your thinking on what the system should refuse? Was there a moment where AI-generated refusal logic was too broad, too narrow, or missed the point?

Commit history requirements:

- ✓ Iterative development — not one or two giant commits
- ✓ Meaningful commit messages that describe what changed and why
- ✓ Evidence that you reviewed and refined AI-generated code, not blind copy-paste

Deliverables

1. GitHub repository with clean commit history
2. README.md — architecture overview (diagram strongly encouraged), setup instructions (docker compose up preferred), API documentation, design decisions and tradeoffs, and cost estimate: estimated cost per query for Features 1 and 2 at 1,000 queries/day and what you would optimize first
3. AI_WORKFLOW.md — see above
4. EVALUATION.md — evaluation results, failure analysis, improvement ideas
5. Video walkthrough (English, 15–20 min) covering: architecture decisions and why; demo of all features including edge cases and a guardrails trigger; evaluation results walkthrough; AI_WORKFLOW.md walkthrough — show your AI development process in action

Bonus (Optional — no extra time required)

In 200 words or fewer, describe how you would add a usage metering layer so Everfit could charge coaches per AI query. No code needed — just the architecture. What would you meter, where would you enforce limits, and how would you handle a coach hitting their limit mid-session?

Materials Provided

Knowledge Base

A zip file containing ~20 markdown documents covering exercise technique guides, training principles (progressive overload, periodization, deload), recovery and nutrition basics, and common workout splits (PPL, Upper/Lower, Full Body). You may supplement with additional content — document what you added and why.

Sample Workout History

A JSON file containing 3 months of sample workout data for 2 fictional users, covering various exercises, progressive overload patterns, and intentional gaps and inconsistencies for edge case testing.